

UBUNTU HOSTS

Event Management Platform

PROJECT PROPOSAL

1. Project Motivation — Why This Project

1.1 The Problem — A Fragmented Landscape

Event organizers across Ethiopia and Africa currently rely on a disconnected set of everyday tools that were never designed to work together. The typical workflow looks like this:

- Excel or Google Sheets — used to maintain attendee lists, often shared manually via email or USB drives
- Telegram or WhatsApp groups — used as a makeshift coordination channel between organizers and staff
- Printed paper forms — used for manual check-ins at event gates, which are then transcribed by hand
- Separate mobile payment apps — used to collect ticket fees with no automatic link to the attendee database
- Phone calls and verbal confirmations — used as the final fallback when digital tools fail or conflict

This fragmentation is not just inconvenient — it creates serious, measurable problems that directly impact the success of every event:

- Double-bookings — when capacity is tracked in a spreadsheet and registrations come in through Telegram, no system prevents two people from claiming the same spot
- Lost registrations — manual data entry across tools introduces transcription errors, leading to attendees showing up with no record of their registration
- Inaccurate capacity counts — because no single tool owns the authoritative attendee list, organizers routinely oversell or undersell events
- Poor attendee experience — attendees receive no automated confirmation, no digital ticket, and often no update if the event changes
- Operational overload — event coordinators spend hours before every event cross-referencing spreadsheets, chasing confirmations, and resolving conflicts that a proper system would prevent automatically

1.2 The Solution — Ubuntu Hosts

Ubuntu Hosts is a consolidated, CRUD-first web platform that replaces this fragmented workflow with a single, authoritative source of truth. Every piece of event data — from creation to registration to capacity tracking — lives in one place, is always accurate, and is accessible in real time by everyone who needs it.

What changes with Ubuntu Hosts:

Organizers create and manage events from one structured interface instead of scattered tools
Attendees register online in seconds with immediate confirmation instead of manual check-ins
Capacity is enforced automatically by the database — overbooking is technically impossible
All event data is persistent, queryable, and accurate — no spreadsheets, no manual reconciliation

1.3 Why "Ubuntu Hosts"

The name Ubuntu comes from the Nguni Bantu philosophy "Umuntu ngumuntu ngabantu" — meaning "I am because we are." It reflects the platform's core purpose: events are fundamentally communal experiences. Ubuntu Hosts exists to make those experiences easier to organize, easier to attend, and more reliable for everyone involved. The word "Hosts" speaks directly to the organizer — the person who brings a community together.

2. Project Goals and Objectives

The following five goals define the measurable success criteria for the Version 1 launch of Ubuntu Hosts. Each goal includes a clear metric, a validation method, and the rationale for why it matters.

01

Event Creation in Under 2 Minutes

An organizer must be able to open the event creation form, fill in all 5 mandatory fields (Title, Date, Time, Location, Description), and have the event saved to the database in under 2 minutes from start to finish.

Why it matters: The primary value proposition of Ubuntu Hosts is speed and simplicity. If creating an event takes longer than the current manual process, organizers have no reason to adopt the platform.

Validation: 10 timed test runs by different users. All 10 must complete within the 2-minute threshold for this goal to pass.

02

Zero-Friction RSVP in 3 Steps or Fewer

An attendee must be able to find an event, register their attendance, and receive confirmation in 3 steps or fewer. No account creation should be required for basic event registration.

Why it matters: Attendee drop-off increases dramatically with every additional step in a registration flow. A 3-step maximum ensures the highest possible conversion from visitor to confirmed attendee.

Validation: Moderated user testing with 10 participants. 9 out of 10 must complete registration without asking for help or abandoning the flow.

03

Zero Overbooking — Absolute Capacity Accuracy

The system must enforce event capacity limits with zero tolerance for overbooking. When an event reaches its maximum registered attendee count, the registration endpoint must automatically reject all further registrations.

Why it matters: Overbooking destroys trust. An attendee who arrives at an event with a valid registration only to be turned away at the door will never use the platform again.

Validation: Concurrent load testing — 50 simultaneous registration requests fired at a near-full event. Zero registrations must be accepted beyond the capacity limit under any condition.

04

99%+ CRUD Reliability Under Normal Load

All five core API operations (Create event, Read all events, Read one event, Update event,

Delete event) must succeed at a rate of 99% or higher under expected production load conditions.

Why it matters: Reliability is the foundation of any data platform. A single failed event creation during a live registration campaign can result in lost bookings, frustrated organizers, and reputational damage.

Validation: Automated API test suite executing 500 requests across all endpoints under simulated load. Pass rate must be 99% or higher.

05

All API Endpoints Respond Within 300ms

Every API endpoint must return a complete response within 300 milliseconds under standard conditions. This includes all read, write, update, and delete operations against the PostgreSQL database.

Why it matters: Response time is perceived quality. Users interpret slow responses as unreliability. 300ms is the industry threshold below which interactions feel instantaneous.

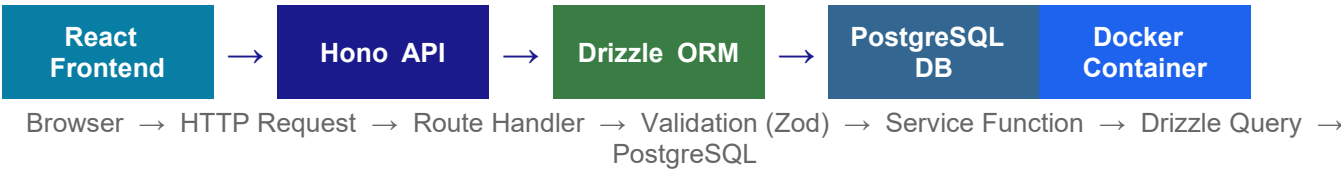
Validation: API performance benchmarking tool measuring p95 response time across all endpoints under standard load. All must fall below the 300ms threshold.

3. Technical Architecture and Justification

Ubuntu Hosts is built on a deliberately minimal, TypeScript-first stack. Every tool was selected to serve a specific architectural purpose — no technology was included out of habit or trend. The result is a stack that is fast to develop, easy to maintain, and built to scale.

3.1 Architecture Overview

The platform follows a Modular Monolith architecture — a single deployable backend application organized into independent feature modules (events, users, tickets, payments). This gives the development team the organizational clarity of microservices without the operational complexity of managing separate services during initial development.



3.2 Stack Breakdown — Full Justification

Layer	Technology	Justification
Language	TypeScript	Static type-checking across the full codebase. Shared types between frontend and backend prevent data shape mismatches. Errors are caught at compile time, not in production.

Layer	Technology	Justification
Frontend	React	Component-driven architecture enables reusable, composable UI elements. The virtual DOM ensures efficient re-renders. Industry-standard ecosystem with excellent TypeScript support.
Runtime	Bun	All-in-one runtime, bundler, test runner, and package manager. Bun is 3–5x faster than Node.js at package installation and startup. Reduces local development friction significantly.
Backend	Hono	Ultra-lightweight web framework (~14kb) that runs natively on Bun. Minimal overhead. Clean, expressive routing API that maps directly to RESTful resource patterns. Built-in middleware support.
Database	PostgreSQL	Production-grade relational database with 30+ years of stability. Enforces data integrity through foreign keys, unique constraints, and transactions. ACID-compliant — critical for registration and capacity tracking.
ORM	Drizzle ORM	TypeScript-first ORM with zero runtime overhead. Queries stay close to raw SQL — developers understand exactly what hits the database. Schema defined in TypeScript so DB structure and app types stay in sync automatically via drizzle-kit.
Validation	Zod	Runtime schema validation that integrates directly with TypeScript inference. All incoming request data is validated at the API boundary before touching the database. Clear error messages returned to the client on failure.
Container	Docker	Runs PostgreSQL in an isolated, reproducible container. Every team member runs the same database version with identical configuration regardless of their operating system. Eliminates "works on my machine" database issues.
Payment	Chapa	Ethiopian payment gateway used to process ticket purchases for paid events. Chapa supports local payment methods including Telebirr, CBE Birr, and major bank cards — making it the ideal choice for an Ethiopia-focused platform. Free events bypass this layer entirely.

4. Database Schema and Data Model

The database is the foundation of Ubuntu Hosts. Every piece of event and registration data flows through a PostgreSQL database managed by Drizzle ORM. The schema is designed to be simple, normalized, and strictly typed.

4.1 Events Table

The events table is the core entity of the platform. Every other table relates back to it.

Column	Type	Constraint	Description
id	SERIAL	PRIMARY KEY	Auto-incrementing unique identifier for each

Column	Type	Constraint	Description
			event
title	TEXT	NOT NULL	The public name of the event (e.g. "Anbesa Beer Festival 2026")
date	TEXT	NOT NULL	The date the event takes place (format: YYYY-MM-DD)
time	TEXT	NOT NULL	The start time of the event (format: HH:MM)
location	TEXT	NOT NULL	The physical or virtual location where the event is held
description	TEXT	NOT NULL	A full description of the event visible to potential attendees
capacity	INTEGER	DEFAULT NULL	Maximum number of attendees allowed. NULL means unlimited
ticket_price	DECIMAL	DEFAULT 0.00	Price of the ticket in ETB. A value of 0.00 means the event is free. Any value above 0 triggers the Chapa payment flow during registration.
created_at	TIMESTAMP	DEFAULT NOW ()	Automatically recorded when the event row is created

4.2 Registrations Table

The registrations table records every attendee who has signed up for an event. It links back to the events table through a foreign key.

Column	Type	Constraint	Description
id	SERIAL	PRIMARY KEY	Auto-incrementing unique identifier for each registration
event_id	INTEGER	FK → events.id	Links the registration to a specific event
attendee_name	TEXT	NOT NULL	Full name of the person registering for the event
attendee_email	TEXT	NOT NULL	Email address of the attendee for confirmation
registered_at	TIMESTAMP	DEFAULT NOW ()	Automatically recorded when the registration is created

5. How the App Works — User Flows

Ubuntu Hosts serves two primary user types: Organizers who create and manage events, and Attendees who discover and register for events. Below are the step-by-step flows for each core interaction.

5.1 Organizer Flow — Creating an Event

Step	Action	What Happens in the System
1	Open the platform	Organizer navigates to the Ubuntu Hosts web app in their browser. The React frontend loads and displays the dashboard.
2	Click "Create Event"	A form component renders with input fields for Title, Date, Time, Location, Description, and optional Capacity.
3	Fill in the form	Organizer types the event details. Zod validation runs on the frontend, highlighting any missing or invalid fields in real time.
4	Submit the form	React sends a POST request to /events with the form data as a JSON body in the HTTP request.
5	API receives request	Hono receives the POST /events request. The zValidator middleware runs the Zod schema check on the request body.
6	Data is validated	If any mandatory field is missing or invalid, Hono immediately returns a 400 error with a clear message. No database call is made.
7	Service function runs	If validation passes, the createEvent() service function is called. Drizzle ORM executes an INSERT query against the events table in PostgreSQL.
8	Response returned	PostgreSQL saves the row and returns the newly created event including its auto-generated ID and created_at timestamp.
9	Success shown	The frontend receives a 201 Created response and displays a success message. The new event appears in the event list immediately.

5.2 Attendee Flow — Registering for an Event

Step	Action	What Happens in the System
1	Browse events	Attendee opens the platform. React fetches GET /events from the Hono API. A list of all available events is displayed.
2	Select an event	Attendee clicks on an event. React fetches GET /events/:id and displays full event details including remaining capacity.
3	Click "Register"	A registration form appears with fields for Name and Email. No account creation is required.
4	Submit registration	React sends POST /registrations with the event ID, name, and email as the request body.
5	Capacity check	The service function checks current registration count against the event capacity before inserting. If full, a 409 Conflict error is returned.

Step	Action	What Happens in the System
5b	Payment required?	If the event has a ticket_price above 0, the attendee is redirected to the Chapa payment page. Free events skip this step entirely and go straight to step 6.
5c	Chapa payment	Attendee completes payment via Telebirr, CBE Birr, or bank card on the Chapa-hosted payment page. Chapa sends a webhook callback to Ubuntu Hosts confirming payment success or failure.
6	Registration saved	If payment is confirmed (or event is free), Drizzle inserts a row into the registrations table with payment status recorded.
7	Confirmation shown	Attendee sees a success message confirming their registration. The event's remaining capacity count updates automatically.

6. API Endpoints — Full CRUD Reference

All endpoints follow RESTful conventions and return JSON. Every write operation (POST, PUT) passes through Zod schema validation before reaching the database. Error responses include a descriptive message field to aid debugging.

6.1 Events Endpoints

Method	Endpoint	Description	CRUD	Status Code
POST	<code>/events</code>	Create a new event. Requires all 5 mandatory fields in the JSON body.	Create	201 Created
GET	<code>/events</code>	Retrieve all events from the database as an array.	Read	200 OK
GET	<code>/events/:id</code>	Retrieve a single event by its unique numeric ID.	Read	200 OK / 404
PUT	<code>/events/:id</code>	Update one or more fields of an existing event. All fields are optional.	Update	200 OK / 404
DELETE	<code>/events/:id</code>	Permanently delete an event. Returns the deleted event object.	Delete	200 OK / 404

6.2 Registrations Endpoints

Method	Endpoint	Description	CRUD	Status Code
POST	<code>/registrations</code>	Register an attendee for an event. Requires event_id, attendee_name, attendee_email.	Create	201 Created
GET	<code>/registrations/:eventId</code>	Get all registrations for a specific event by event ID.	Read	200 OK / 404
DELETE	<code>/registrations/:id</code>	Cancel a registration by its unique ID.	Delete	200 OK / 404

7. Scope

Defining what is explicitly excluded from Version 1 is as important as defining what is included. A focused initial scope ensures the team delivers a polished, reliable core product rather than a half-finished feature-heavy one.

In Scope — Version 1 Features
✓ Full CRUD for event management (Create, Read, Update, Delete)
✓ Event creation form with 5 mandatory fields: Title, Date, Time, Location, Description
✓ Event listing page — browse all available events
✓ Single event detail view with full information
✓ Basic attendee RSVP registration (name + email)
✓ Capacity tracking — automatic closure at maximum
✓ Input validation on all endpoints using Zod
✓ RESTful JSON API with documented endpoints
✓ PostgreSQL database with normalized schema
✓ Docker-based local development environment